Zipper Algebras: Mathematical Context and Software Specification

29 October 2026

1 Mathematical purpose

Zipper algebras arise in the study of quotients of braid group algebras and representations motivated by quantum topology. Bigelow introduced them in [1, §8, pp. 297–298], asking whether the three-strand slider vanishes and which additional relations yield finite-dimensional quotients (Questions 6 and 7). The Hecke and BMW quotients provide familiar points of comparison. The research programme developed in [2] constructs a family of untwisting quotients and asks how their algebraic structure depends on the untwisting polynomials.

The zipper algorithm and its finite-generation consequence give the broader mathematical setting for this software. The computational questions include whether one slider or untwisting relation follows from others, how presentations change with the untwisting choice, and how to obtain explicit identities that prove those dependencies. The package supports these questions through exact noncommutative algebra, proof expansion, two-sided ideal certificates and rational-function reconstruction. The original Gröbner-basis and cofactor discovery used `f4ncgb`, by Heisinger and Hofstadler [5].

This reference first defines the algebras and states the relevant thesis result, then specifies the implemented presentations and verification requirements and conclusions. The geometric algorithm and its diagrams have their own exposition in [2, §3.4].

2 Ambient algebras and conventions

Put $P = \mathbb{Q}[q,t]$ and $K = \mathbb{Q}(q,t)$, where q, t are algebraically independent central indeterminates. For $n \geq 2$, use the free associative algebra

$$F_n = P\langle x_1, \ldots, x_{n-1}, y_1, \ldots, y_{n-1}\rangle.$$

A word is an ordered tuple of indices in that displayed generator order, starting at one. The empty tuple represents the unit. Concatenation denotes multiplication, in the order written. In the braid group algebra, x_j maps to σ_j and y_j to σ_j^{-1}. Over a commutative coefficient ring R, the corresponding braid group algebra is $R[B_n]$, with q a unit when using the untwisting hypotheses below.

The presentation explicitly includes the inverse relations $x_j y_j - 1$ and $y_j x_j - 1$, adjacent braid relations and far commutations, together with redundant inverse or mixed-sign versions used by the reduction engine. Let J_n be the ideal generated by the braid and inverse relations. Passing from F_n to F_n/J_n identifies freely different words representing the same braid. Consequently, an

identity in the braid group algebra need not be a literal free-algebra identity. The maintained verifier checks the latter. A certificate using braid simplification must supply the corresponding braid/inverse terms or use a separately specified group-algebra checker.

3 Sliders, zipper algebras and untwisting

We use the notation of [2, Definitions 2.6, 2.9 and 3.1]. For the braid formulas below, expand each displayed abbreviation into its specified signed word, then replace σ_j by x_j and σ_j^{-1} by y_j to obtain the corresponding free-algebra polynomial used by the package. This translation preserves word order and applies no braid or inverse relations. The images in $R[B_n]$ and the chosen free-algebra representatives must therefore be distinguished.

3.1 Braid words and sliders

Following the thesis's notation for fat crossings, put

$$\varsigma_{k,j} = \sigma_j\sigma_{j+1}\cdots\sigma_{j+k-1}, \qquad \bar{\varsigma}_{k,j} = \sigma_j^{-1}\sigma_{j+1}^{-1}\cdots\sigma_{j+k-1}^{-1},$$
$$\varsigma_{k,j}^{-1} = \sigma_{j+k-1}^{-1}\cdots\sigma_j^{-1}, \qquad \bar{\varsigma}_{k,j}^{-1} = \sigma_{j+k-1}\cdots\sigma_j.$$

Here k counts crossings: the supported block has $k+1$ strands, with $1 \le j \le n-k$. For $k \ge 2$, $\bar{\varsigma}_{k,j}$ and $\varsigma_{k,j}^{-1}$ have different word orders. For an i-strand slider starting at j, the recursion is

$$X_{2,j} = q\sigma_j^{-1} + (1-q) - \sigma_j,$$
$$X_{i,j} = (q^{i-1}\bar{\varsigma}_{i-1,j} - \varsigma_{i-1,j})X_{i-1,j} \quad (3 \le i \le n),$$

with $1 \le j \le n-i+1$, and $X_i = X_{i,1}$. These are explicit elements of the braid group algebra. Bigelow's barred indexed word denotes the inverse of the entire indexed product. Translating that abbreviation gives the same recursion [1, p. 298].

The package's slider dictionary is named `S`. Its entry `S[(i,j)]` is the chosen free-algebra representative of $X_{i,j}$. The principal translations are:

Mathematical notation	Code representation
σ_j	`xj`, generator index j
σ_j^{-1}	`yj`, generator index $n-1+j$
$X_{i,j}$	`S[(i,j)]`
$X_i = X_{i,1}$	slider target `X_i`
$\varsigma_{i-1,j}$	ascending product `xj` through `x(j+i-2)`
$\bar{\varsigma}_{i-1,j}$	ascending product `yj` through `y(j+i-2)`

3.2 The zipper algebra

The level-two zipper relator is

$$Z_{2,j} = \big(q\sigma_{j+1}^{-1} + 1 - q - \sigma_{j+1} - q\bar{\varsigma}_{2,j} + \varsigma_{2,j}\big)X_{2,j}.$$

For $i \ge 3$, the higher zipper relator is

$$Z_{i,j} = \big(q^{i-1}\bar{\varsigma}_{i-1,j+1} - \varsigma_{i-1,j+1} - q^{i-1}\bar{\varsigma}_{i,j} + \varsigma_{i,j}\big)X_{i,j}.$$

Here $3 \le i \le n-1$ and $1 \le j \le n-i$; for level two, $1 \le j \le n-2$. In particular, the two-strand formula is separate and includes its scalar term $(1-q)X_{2,j}$. The package emits the chosen free-algebra representatives of these relators.

The zipper algebra is

$$\mathcal{Z}_n(R,q) = R[B_n]/\langle Z_{i,1} : 2 \le i \le n-1 \rangle.$$

The builder also emits their shifted versions. These generate the same ideal: conjugation by $\sigma_1 \cdots \sigma_{n-1}$ shifts each σ_k to σ_{k+1} while $k \le n-2$, so successive conjugations translate each supported block.

Imposing $X_2 = 0$ gives the Hecke quotient: indeed $\sigma_1 X_2 = -(\sigma_1 - 1)(\sigma_1 + q)$. For $n \ge 3$, the quotient of the zipper algebra by $X_3 = 0$ and $(\sigma_1 - t)X_2 = 0$ is the BMW braid-generator presentation described in [1, §8] and [2, §3.2]; see also [3, 4]. The parameter comparison with the conventional BMW generators is discussed below.

3.3 The family of polynomial untwisting quotients

The thesis defines the lasso full-twists by

$$\Theta_{k,j} = \varsigma_{k,j}\bar{\varsigma}_{k,j}^{-1}, \qquad \Theta_2 = \sigma_1, \qquad \Theta_i = \Theta_{i-1,1} \quad (i \ge 3).$$

Thus $\Theta_3 = \sigma_1\sigma_2^2\sigma_1$. The first subscript of $\Theta_{k,j}$ counts crossings in its two factors, while the subscript of Θ_i denotes the strand count. Consequently, the shifted full-twist on the support of $X_{i,j}$ is $\Theta_{i-1,j}$ for $i \ge 3$. For monic polynomials $p_2, \ldots, p_n \in R[\lambda]$ with unit constant terms, form the two-sided quotient

$$\mathcal{Z}_n^{(p)} = \mathcal{Z}_n(R,q)/\langle p_2(\sigma_1)X_2, p_3(\Theta_3)X_3, \ldots, p_n(\Theta_n)X_n \rangle.$$

This is the class of algebras in [2, Definition 2.9], allowing an independent untwisting polynomial at every level.

Finite-generation theorem (Moos). *With $q \in R^\times$ and the above monicity and unit-constant-term hypotheses, $\mathcal{Z}_n^{(p)}$ is finitely generated as an R-module. In particular, over a field it is finite-dimensional.*

This is the finite-generation formulation of [2, Theorem 3.41, p. 62]. The theorem follows from the zipper algorithm's layered-zipper spanning result (Theorem 3.39): there are finitely many combinatorial layer types up to twisting, and the polynomial relations bound the remaining twist exponents. Monicity reduces positive powers; a unit constant term also permits reduction of negative powers. Thus the theorem supplies a family to study as the polynomials vary. It does not assert that the resulting spanning set is a basis, determine the dimension, or establish semisimplicity.

3.4 Presentations implemented by the software

The function `build_relations_ring` is in `src/zipper_algebra/relations.py`; its front end is `Presentation` in `src/zipper_algebra/presentation.py`. A presentation records the strand count, an untwisting preset or explicit per-level polynomials, powers, level limits, imposed zero sliders and named additional relations. Its symbolic relation content is hashed; readable labels alone do not identify an algebra. The Python constructor `Presentation.from_untwistings` accepts independent rational-coefficient polynomials at selected levels. It emits $p_2(\sigma_j)X_{2,j}$ at level two

and $p_i(\Theta_{i-1,j})X_{i,j}$ at higher levels, including every valid shift. An omitted level supplies no untwisting relation. The existing presets retain their chosen free-algebra representatives and presentation hashes.

In the canonical untwisting presentation, the relations are the chosen free-algebra representatives of

$$T_{2,j} = (\sigma_j - t)X_{2,j}, \qquad T_{i,j} = \big(\bar{\varsigma}_{i-1,j}^{-1} - q^{i-1}\varsigma_{i-1,j}^{-1}\big)X_{i,j} \quad (i \geq 3).$$

In the braid quotient, left multiplication by $\varsigma_{i-1,j}$ converts the higher relation to $(\Theta_{i-1,j} - q^{i-1})X_{i,j} = 0$. This equivalence uses invertibility; the emitted relation and this full-twist form have different chosen representatives in the free algebra.

3.5 Experimental parameter choices

The builder also implements a level-two power relation $(\sigma_j^a - t)X_{2,j}$, the squared relation $(\sigma_j - t)^2 X_{2,j}$, and a historical option $(\sigma_j^2 - t^2)X_{2,j}$. The twist-power parameter affects level three only: for exponent $m > 1$, the emitted relation is the chosen representative of $(\Theta_{2,j}^m - q^{2m})X_{3,j}$. At higher levels the canonical relation remains. The established `Presentation` interface accepts rational coefficients; the domain-bound interface below also accepts parameter-dependent coefficients. Nonmonic inputs and zero constant terms are accepted as raw relations, without normalization or a finiteness verdict. The theorem's unit hypotheses must be checked in the selected coefficient domain, including after specialization. Exact input and serialization examples are in `docs/POLYNOMIAL_UNTWISTINGS.md` and `docs/PARAMETER_FIELDS.md`. No structural theorem about a variant follows simply from choosing a flag.

The public presentation object rejects unknown untwisting choices, noninteger strand counts and powers, level bounds outside $2, \ldots, n$, repeated additional relation labels and targets requiring more strands than are present. Numeric coefficients use either $\mathbb{Q}$ or $\mathbb{Z}/m\mathbb{Z}$. Calling the latter a field requires the additional hypothesis that m is prime; the coefficient backend does not perform a primality proof. Nonunits cannot be inverted. The new field interface below verifies positive characteristic separately.

4 Exact parameter fields and domain-bound identities

The classes in `src/zipper_algebra/fields.py` implement $k(z_1, \ldots, z_r)$ for $k = \mathbb{Q}$ or a checked prime field, including $r = 0$. The ordered parameter list and characteristic belong to the domain. Sparse polynomials have nonnegative exponent tuples of the declared arity. A field element stores a numerator and nonzero denominator; equality is checked by cross multiplication. No polynomial gcd or factorization is needed for this equality test. Equal fractions may retain different literal representations.

Specialization evaluates a specified finite collection of stored expressions where their denominators are defined and nonzero. Rational constants must also be defined in the selected characteristic. This operation is not a map from the entire function field to a fibre. For example, $(s^2-1)/(s-1) = s+1$ in $\mathbb{Q}(s)$, but the former stored representative excludes $s = 1$. Arithmetic can change the representative and its recorded excluded locus, including by cancellation to zero. Generic degree and dimension need not survive specialization.

The classes in `src/zipper_algebra/algebra.py` record an explicit field, ordered alphabet and literal ordered relations. Even a zero free polynomial retains its complete parent. The constructor

`AlgebraPresentation.zipper` substitutes the chosen q into the existing bare zipper emitter, then appends independent polynomial untwistings at all valid shifts. No omitted level is inserted. The versioned JSON identity and its SHA-256 record the field, alphabet and literal coefficients, including denominator data. Equal fractions can conservatively give different literal presentation identities.

A `CofactorIdentity` verifies

$$f = \sum_{\nu} L_\nu r_{i_\nu} R_\nu$$

by exact free-algebra arithmetic over the declared field. Its factors may be free polynomials, and its relation references are zero-based indices. Both the parent and the full presentation hash are checked. A successful check establishes membership over that field; it does not establish membership over a smaller coefficient ring or at a pole. This JSON format is separate from the established symbolic q,t text-certificate format described below. Solver production and reconstruction for arbitrary parameter fields remain further integration work.

4.1 Two-strand acceptance controls

The following calculations give independently known answers for testing exact cofactor and representation operations. They are small controls for the parameter-field interface; the generic families remain the research setting. Put $x = x_1$, $y = y_1$, $X = qy + (1-q) - x$, $r_0 = xy - 1$ and $r_2 = p(x)X$. For any polynomial p with central coefficients, set $F(x) = p(x)(x-1)(x+q)$. There is a literal free-algebra identity

$$F(x) = q\,p(x)r_0 - xr_2.$$

Modulo $xy = yx = 1$, conversely $r_2 = -yF(x)$. Thus the two-strand quotient is $k[x, x^{-1}]/(F)$. If F is monic of degree N with nonzero constant term, the words $1, x, \ldots, x^{N-1}$ span. The companion matrix of F, together with its inverse, satisfies the defining relations, and these words applied to the first coordinate vector give the coordinate basis. This proves independence and hence dimension N. In particular, a monic degree-d untwisting with nonzero constant term gives dimension $d+2$ when $q \neq 0$.

The executable examples in `examples/polynomial_quotients.py` check this cofactor and cyclic representation. Over $\mathbb{Q}(q,a,b)$ they use $p(x) = x^2 + ax + b$ and the independently expanded polynomial

$$F(x) = x^4 + (a+q-1)x^3 + (b + a(q-1) - q)x^2 + (b(q-1) - aq)x - bq,$$

giving dimension four. A prime-field specialization is also checked. For $q = 2$ and $p = (x-u)^2$ over $\mathbb{Q}(u)$, the dimension is again four. Writing $h = (x-1)(x+2)$, the element $(x-u)h$ is nonzero and square-zero: its square is Fh, and the companion representation verifies nonvanishing.

Two exceptional-fibre examples distinguish different mechanisms. For $q = 2$ and $p_s = sx^2 + x + 1$, the generic dimension is four, while at $s = 0$ it is three. Dividing by s to obtain a generic monic polynomial would exclude that fibre. For $p_s = x^2 - s$ the generic dimension is four, but the fibre at $s = 0$ has dimension two: x^2 is invertible in the Laurent algebra, leaving only $(x-1)(x+2)$. The examples certify removal of these invertible factors using explicit inverse-relation cofactors. These conclusions use the exact two-strand calculation, rather than assuming the full untwisting theorem's hypotheses survive specialization.

5 Complete Gröbner bases and certified normal forms

Let $F = k\langle x_1, \ldots, x_m\rangle$, where k is one of the exact fields above, and let $I = (r_1, \ldots, r_s)$. The explicit verifier in `verification/groebner.py` accepts a candidate family of nonzero polynomials $h_1, \ldots, h_N$, bound to the full presentation identity. Each h_j carries a local identity of the form

$$h_j = \sum_{\nu} c_\nu u_\nu r_{i_\nu} v_\nu + \sum_{\mu} d_\mu a_\mu h_{\ell_\mu} b_\mu, \qquad c_\nu, d_\mu \in k,$$

where all contexts are words. References to other candidate polynomials form a directed graph. All references, including zero-coefficient references, are validated, and the graph must be acyclic. Checking the displayed equalities and inducting in dependency order proves $h_j \in I$. Shared subproofs need not be expanded into cofactors of the original relations.

The order is fixed degree-lexicographic order, with lexicographic comparison using the declared generator indices. Leading words are computed from the polynomials, and nonzero leading coefficients are inverted in k. Every resulting rewrite decreases this admissible order. The verifier checks that each original r_i reduces to zero, proving $I \subseteq (h_1, \ldots, h_N)$; the derivations prove the reverse inclusion. It then reduces every inclusion ambiguity, including equal leading words, and every proper nonempty suffix-prefix overlap for all ordered pairs, including self-overlaps. When every remainder vanishes, Bergman's Diamond Lemma [6, Theorem 1.2] gives confluence. The irreducible words form a k-basis of F/I, and reduction gives a unique normal form. Consequently, a zero normal form proves membership in the original ideal and a nonzero normal form proves nonmembership over the stated field.

Optional rule pruning uses leading-word divisibility to select reduction rules. It does not omit validation of the derivation graph, reduction of the original inputs or resolution of ambiguities for the retained rules. A candidate polynomial proved to be a nonzero constant gives $I = F$ and dimension zero.

Only after completion is established does the verifier enumerate irreducible words. If this enumeration finishes, its length is $\dim_k(F/I)$. Exceeding the word-count limit returns an undetermined dimension, not an infinity verdict. Exceeding the rewrite/ambiguity work budget raises `VerificationLimit`, without reporting a completed verification. A returned `VerifiedReduction` retains immutable defining data; later normal-form calls use the remaining work budget and reject elements with a different parent, including zero elements.

The JSON schema `zipper-groebner-certificate-v1` stores the candidate polynomials, local `ProofTerm` data and presentation hash. The numeric f4ncgb decoder `certificate_from_solver` in `verification/f4ncgb.py` supplies candidates, not a completion verdict. The established discovery records continue to report `basis_complete=False`; their membership validation is separate from this explicit completion operation. The four-strand example below has a separate local Singular producer; this decoder does not integrate arbitrary parameter-field producers. GBNP's `IsGrobnerBasis` provides an independent completion check for its supported fields; its quotient basis and dimension functions require a genuine Gröbner basis [7]. A basis check alone does not identify its ideal with the original defining ideal. The API and resource limits are detailed in `docs/GROEBNER_VERIFICATION.md`.

6 Finite word-basis certificates and regular actions

Let $A = F/I$, where $F = K\langle g_1, \dots, g_m\rangle$ and $I = (r_1, \dots, r_s)$ over one of the declared exact fields. A finite word-basis certificate supplies distinct words $B = (b_0 = 1, b_1, \dots, b_{d-1})$ and sparse matrices $M_1, \dots, M_m \in \mathrm{Mat}_d(K)$. The column convention is that column j of M_a gives the proposed coordinates of $g_a b_j$.

Proposition 1. *Suppose explicit ideal-membership identities prove*

$$g_a b_j - \sum_{k=0}^{d-1} (M_a)_{kj} b_k \in I \quad (1 \le a \le m,\ 0 \le j < d),$$

and suppose that

$$r_i(M_1, \dots, M_m) = 0 \quad (1 \le i \le s), \qquad b_j(M_1, \dots, M_m) e_0 = e_j \quad (0 \le j < d).$$

Then the images of B are a basis of A, $\dim_K A = d$, and the M_a are its left regular generator actions in this basis.

Proof. The span of the images of B contains the unit and, by the ideal identities, is stable under left multiplication by every generator. Induction on word length proves spanning. The matrix relations give a representation $\rho : A \to \mathrm{End}_K(K^d)$. If $\sum_j c_j b_j = 0$ in A, applying ρ and evaluating on e_0 gives $\sum_j c_j e_j = 0$, so all $c_j = 0$. This proves independence. The certified column identities then identify the matrices with the regular actions. □

A cyclic representation alone can describe a proper quotient; the spanning identities supply the upper dimension bound. Spanning identities alone can describe dependent words; the cyclic representation supplies independence. Every defining matrix relation is checked on every coordinate vector, not only on e_0. In dimension zero the separate requirement is an exact identity $1 \in I$; empty matrices give no such evidence by themselves.

The immutable `WordBasisCertificate` records the words, sparse generator actions, action-node references and a shared `DerivationGraph`. Local graph identities are checked on the full acyclic graph, including raw references with zero coefficients; zero polynomial nodes are allowed. The JSON schemas are

- `zipper-word-basis-certificate-v1`;
- `zipper-generator-action-v1`;
- `zipper-derivation-graph-v1`.

Existing literal presentation hashes and Gröbner-certificate encodings are preserved. A completed basis can help produce these witnesses, but completeness is not a hypothesis of this finite-basis verification theorem.

The verifier `verify_word_basis` returns an immutable `VerifiedFiniteAlgebra` with exact coordinates and products. Generator actions remain sparse. Products are evaluated on demand from the named basis words, without constructing a cubic-size multiplication table. Matrix relation evaluation shares a noncommutative Horner graph so that cancellation can occur before further matrix applications. For a parameter field $K = \mathrm{Frac}(R)$ with polynomial ring $R = k_0[z_1, \dots, z_r]$, the

working evaluation writes each generator matrix as $M_a = A_a/d_a$, where A_a has polynomial entries and $d_a \in R$ is nonzero. A Horner node representing $P_n(M)$ is assigned a common denominator D_n. For a seed vector U/S, it evaluates the polynomial vector

$$V_n = D_n P_n(M)U, \qquad P_n(M)(U/S) = V_n/(D_n S).$$

The D_n clear the node's constant coefficient and the coefficients multiplying $A_a V_{\text{child}}/(d_a D_{\text{child}})$. Their construction uses common denominators and exact polynomial quotients before repeated matrix-vector arithmetic. The inner evaluation then uses polynomial addition and multiplication, avoiding a fraction gcd at each accumulation. Arbitrary polynomial denominators are supported. All conclusions remain over K; this computational clearing does not assert an R-basis or specialize at zeros of the denominators.

Explicit operation, dimension and proof-node limits raise `VerificationLimit` on exhaustion. Subsequent coordinate and product calculations each have their own operation budget. The default finite-algebra operation budget is 10^7; the separate derivation-only checker retains its $2 \cdot 10^6$ default.

Parameter-field computations use an internal normalized working representation based on SymPy's sparse rational-function fields [10], installed through the optional `symbolic` extra. Ground-field arithmetic needs no optional dependency. This working arithmetic does not rewrite the literal input coefficients or their presentation hash. A converted output coefficient may have cancelled denominator factors and therefore its own conditions for any specified specialization. The original literal denominator data remain part of the input. API details and limits are in `docs/FINITE_ALGEBRAS.md`.

7 Symbolic q,t certificate data and exact conclusion

Let $r_1, \dots, r_s \in F_n$ be the ordered defining relations of a specified presentation. A certificate for $f \in F_n$ is a finite list of terms $(u_\nu, i_\nu, v_\nu, N_\nu, e_\nu)$, together with an ordered family of nonzero polynomials $a_1, \dots, a_h \in \mathbb{Z}[q,t]$. Here u_ν, v_ν are words, i_ν names an actual relation, $N_\nu \in P$, and $e_\nu \in \mathbb{N}^h$. The intended identity is

$$f = \sum_\nu \frac{N_\nu}{D_\nu} u_\nu r_{i_\nu} v_\nu, \qquad D_\nu = \prod_{k=1}^{h} a_k^{e_{\nu k}}, \tag{1}$$

in $K\langle x_1, \dots, y_{n-1}\rangle$. Atom irreducibility is unnecessary for checking this identity; it is relevant to reconstruction efficiency. The atom search bounds do not limit the exponents in a composed certificate.

Define $E_k = \max_\nu e_{\nu k}$, with zero maxima for an empty certificate, and $M = \prod_k a_k^{E_k}$. Since P is an integral domain and each atom is nonzero, $M \neq 0$. The exact checker compares the polynomials

$$Mf \quad \text{and} \quad \sum_\nu N_\nu \left(\prod_k a_k^{E_k - e_{\nu k}} \right) u_\nu r_{i_\nu} v_\nu. \tag{2}$$

Equality proves (1) and hence membership of f in the two-sided ideal generated by the relations over K. It also proves membership in the corresponding localization of P, inverting the used atoms. A specialization of the rational identity is available wherever the used polynomial factors are nonzero. Atom names in a serialized certificate are formal labels; their explicit polynomial definitions determine the specialization condition. Verification reports display those polynomial

factors. The polynomial identity (2) remains meaningful at their zeros, but need not imply the specialized membership statement there.

The certificate boundary rejects zero atoms, nonintegral atom coefficients, negative or noninteger polynomial exponents, denominator vectors of the wrong length, inexact numerical coefficients, out-of-range word indices and unknown relation labels. Both verifiers validate the supplied presentation identity. Objects are revalidated at the checking boundary because their data are mutable. A malformed certificate raises a validation error. A well-formed false identity returns a report with `pass=False`.

Legacy files without a presentation hash require an explicitly supplied presentation. Their historical origin does not replace verification against those actual relations. New certificates should retain the presentation hash and explicit target. Named sliders, additional named targets and direct polynomial targets share the same validated target resolution.

8 Independent polynomial identity testing

The second checker evaluates the cleared identity on a product grid of distinct rational integers. For each parameter, its bound includes both the target degree plus the clearing-factor degree and the degrees of every cofactor term. If the bounds are (d_q, d_t), a grid with $(d_q + 1)(d_t + 1)$ points suffices.

Proposition 2. *If a polynomial in $\mathbb{Q}[q, t]$ of bidegree at most (d_q, d_t) vanishes on such a product grid, it is zero.*

Proof. Fix a grid value of t and apply the univariate root bound in q. Each coefficient in q then vanishes at all $d_t + 1$ values of t, so the same root bound makes each coefficient zero. Apply this to every free-word coefficient. □

The checker evaluates the polynomial product M/D_ν directly, rather than dividing its evaluations. Thus grid points at poles cause no undefined division. Negative numerator exponents are rejected: a Laurent polynomial cannot use this polynomial degree bound without additional clearing. This exact finite-grid argument differs from checking a few selected parameter values. Its arithmetic is separate from the symbolic accumulation, but it shares the presentation builder, input validation and target resolution.

9 Proof expansion and contextual composition

A basis proof is an acyclic directed graph expressing each basis element as a sum of contextual multiples of input relations and other basis elements. Forward references are allowed; cycles and missing dependencies are errors. Membership validation checks the local polynomial identities in dependency order, without expanding shared subproofs. Explicit cofactor composition may subsequently expand these derivations. Substitution respects multiplication order. If

$$g_k = \sum_\nu c_{k\nu} L_{k\nu} r_{i_{k\nu}} R_{k\nu}, \qquad f = \sum_s m_s u_s g_{k_s} v_s,$$

composition produces terms with coefficient $m_s c_{k_s\nu}$ and contexts

$$(u_s L_{k_s\nu},\ i_{k_s\nu},\ R_{k_s\nu} v_s).$$

No commutation or cancellation of free words is implicit. Repeated uses of one basis element remain separate until their full contexts agree and coefficients can be added. The number of multipliers must equal the number of skeleton steps, and an explicit nonzero skeleton remainder precludes membership composition.

For sampled multipliers, each sample reduction must have the same ordered sequence of (u_s, k_s, v_s) as the anchor, where k_s is the leading-word key rather than the engine's changing numerical index. The data retain one multiplier per occurrence. A sample with a different contextual schedule is excluded. A successful fit is a candidate; the resulting identity still needs exact verification.

10 Exact reconstruction of candidates

10.1 Standalone coefficient fitting

A sample consists of a distinct parameter point in $\mathbb{Q}^2$ and an exact rational value. Repeated locations count once; conflicting values and floating-point coefficients are rejected. For a fixed nonzero denominator proposal and a numerator bidegree bound, interpolation uses the available fitting rows and requires full column rank. A consistent rank-deficient system does not determine a unique numerator and is not completed by arbitrarily setting free variables to zero. Numerator coefficients belong to $\mathbb{Q}$, rather than necessarily $\mathbb{Z}$.

Univariate rational reconstruction fixes a monic denominator, solves the exact linear system, cancels a polynomial greatest common divisor and normalizes the result. The reduced denominator must be nonzero at every supplied point, and the candidate must match every fitting and held-out value. This prevents a spurious cross-multiplied equation $0 = 0$ from concealing an incorrect sample at a pole.

In the standalone fitting API the factor family is proposed from fitting data, starting without predeclared denominator factors. Exact univariate factorization and lifting across parameter lines are tried before bounded polynomial enumeration. The current line-lifting budget allows degree at most four in the other parameter. Varying-factor lifting uses at least five distinct parallel coordinates and reserves two for validation, subject to the requested larger minimum. Repeated univariate factors also require at least five distinct parallel coordinates. These are inference heuristics. The search has explicit finite budgets and can leave unresolved line factors; it does not certify that every generic pole has been found. An explicitly supplied ordered factor family remains a user-selected search constraint.

Held-out locations are reserved before fitting and denominator inference. Their coefficients do not participate in inference of the factor family. A passing held-out check is additional evidence, not a rational-function identity proof. Line restrictions can cause distinct polynomial factors to coincide, so inferred exponents are heuristic proposals. They are validated and have a general-search fallback. For arbitrary declared factors, absence of a cancellable whole factor does not imply coprimality of the numerator and denominator; stronger assertions require irreducible, pairwise nonassociate factors.

Failed predictions, an unavailable common reduction shape and exhaustion of the chosen degree or factor bounds do not show that more data cannot help. The standalone fitting routine reports an inconclusive outcome when it lacks a candidate in its search class. In particular, reducing to a nonzero remainder against an uncertified basis establishes no nonmembership conclusion, even at that specialization. The exception `NeedMoreSamples` reports an inconclusive reconstruction.

Candidate reconstruction is characteristic zero.

10.2 Discovery from contextual input relations

The installed discovery command obtains a candidate basis and a nonexpanded proof graph at a rational anchor. Every numerical basis identity is checked against the actual specialized defining relations. Reducing the target records its ordered contextual uses of the candidate basis. Expanding the relevant proof graph propagates left and right word contexts to the original input relations. Numerical proof coefficients are discarded in this step: they supply neither the generic coefficients nor denominator proposals.

Let the resulting distinct triples be (u_j, i_j, v_j), for $1 \le j \le c$, where r_{i_j} is an original defining relation over $K = \mathbb{Q}(q,t)$. Take the union of all words occurring in $u_j r_{i_j} v_j$ and the target h, and fix an order on that finite set. Relative to that free-word basis, let A have columns $u_j r_{i_j} v_j$, let b be the coordinate column of h, and put

$$M = [A \mid -b].$$

FiniteFlow reconstructs a complete right kernel N of M, which is then verified independently as described below. If a column (z, s) of N has $s \neq 0$, then $Az = bs$, hence

$$h = \sum_{j=1}^{c} \frac{z_j}{s}\, u_j r_{i_j} v_j \quad \text{in } K\langle \mathcal{G} \rangle.$$

A separate free-algebra checker expands the original relations with these coefficients and checks the complete identity before publication. Thus even an error in support construction or matrix assembly cannot be accepted solely because the matrix calculation succeeds.

The anchor graph proposes a finite support; it does not prove that this support suffices generically. A coefficient that vanishes at the anchor can hide a needed context. If every kernel vector has final coordinate zero, this support cannot represent the target, but membership in the full defining ideal remains unresolved. A nonzero target remainder against an uncertified basis likewise establishes no nonmembership result. Support-size, exact-product, reconstruction degree and prime budgets yield an inconclusive outcome upon exhaustion; they are computational limits rather than hypotheses of an accepted identity. The command continues to report `basis_complete=False`.

10.3 Finite-field reconstruction of matrix kernels

Let $K = \mathbb{Q}(t_1, \ldots, t_s)$, let $M \in K^{m \times n}$, and fix the ordered parameters of K. FiniteFlow [9] produces a candidate kernel by solving specialized finite-field systems and reconstructing their rational coefficients. This computation is separate from generating noncommutative word contexts or proving that a matrix is the regular action of an algebra.

A kernel certificate supplies $N \in K^{n \times (n-r)}$, a list J of $n - r$ distinct row indices, and an $r \times r$ minor B of the original matrix. It includes a prime p, a parameter point $a \in \mathbb{F}_p^s$, and an inverse $C \in \mathrm{Mat}_r(\mathbb{F}_p)$ for $B(a)$. Every stored input coefficient used in $B(a)$ must be defined under this specialization: rational constant denominators and evaluated polynomial denominators must be nonzero. The independent verifier checks

$$MN = 0, \qquad N_{J,*} = I_{n-r} \quad \text{over } K, \qquad CB(a) = I_r \quad \text{over } \mathbb{F}_p.$$

The first two identities imply $\operatorname{rank}_K M \le r$, since the columns of N are independent. The last identity implies that the determinant of B is a nonzero rational function and hence $\operatorname{rank}_K M \ge r$. Together they prove the exact generic rank and a basis of the right kernel. Empty identity blocks have their usual meaning for ranks zero and n.

The matrix identity binds the witness to the supplied input. It does not by itself establish how that matrix was obtained from a presentation; that source requires its own finite-basis, action or cofactor checks. Sample agreement, a library success status, and a chosen reconstruction degree bound do not replace these identities. Unlucky primes, points, or learned pivot choices can cause failure or an incorrect candidate; a candidate is accepted only after all exact checks pass. Coefficient bounds, prime budgets and thread counts are computation settings rather than hypotheses of the conclusion.

11 A four-strand square-zero quotient over $\mathbb{Q}(q,t)$

Let A be the four-strand zipper quotient over $k = \mathbb{Q}(q,t)$ with polynomial untwistings

$$p_2(\lambda) = (\lambda - t)^2, \qquad p_3(\lambda) = \lambda - q^2, \qquad p_4(\lambda) = \lambda - q^3,$$

at every valid shift. Thus the two-strand untwisting relation is $(x_j - t)^2 X_{2,j} = 0$, and the higher untwistings are canonical. The polynomials are monic with unit constant terms in k, so the finite-generation theorem above applies. The literal presentation contains 31 ordered relations, including the canonical four-strand relation T_4 in its chosen free-algebra representative. The example constructs `Presentation(4, "square-zero")` and converts its coefficients to k. It therefore uses the canonical higher relations T_3, T_4, rather than the literal full-twist polynomial representatives emitted by `from_untwistings`. Inverse relations identify the ideals of these representatives by invertible left factors, but the free polynomials and literal presentation identities differ. The following are generic conclusions over k:

$$\dim_k A = 186, \qquad X_3 = 0, \qquad w = (x_1 - t)X_{2,1} \neq 0, \qquad wAw = 0.$$

In particular, $J = AwA$ is a nonzero two-sided ideal with $J^2 = A(wAw)A = 0$. Hence A is not semisimple: the nonzero square-zero ideal J supplies the obstruction.

The certificate records 299 auxiliary input-derived polynomials and 148 candidate basis polynomials: 447 nodes with explicit local derivations. The GAP checker requires every input reference in the entire graph to lie among the first 30 relations, excluding T_4. The local identities therefore prove that the candidate ideal lies in the ideal of those 30 inputs. Independently, reduction of all 31 input relations to zero proves the reverse inclusion, and the critical-ambiguity checks establish completion of the 148-polynomial basis. Enumeration gives 186 irreducible words. The nonzero normal form of w proves nonvanishing; the exact normal forms of wbw vanish for all 186 basis words b, proving $wAw = 0$ by linearity. The normal forms of X_3 and T_4 also vanish.

Candidate production uses Singular's letterplace computations [8]; the derivation, completion, dimension and ideal checks use GAP 4.15.1 with GBNP 1.1.0 independently [7]. The input-reference bound and vanishing normal form of T_4 together prove its redundancy. The full 447-node Python Gröbner verification did not finish in a 60-second local probe; that completion route uses Singular and GAP. A separate finite word-basis certificate extends those 447 nodes with 1,116 generator-column identities. The maintained Python finite-basis verifier checked all 1,563 nodes, the 5,766 original matrix-relation columns and 186 cyclic-coordinate identities, independently certifying dimension 186 and the sparse regular generator actions without a completion hypothesis.

Its six generator matrices have 14,511 nonzero entries. Commands and the recorded verification counts are in `docs/FINITE_ALGEBRAS.md`. This result concerns the generic parameter field, with no specialization involved. Commands, source roles and generated proof data are described in `docs/FOUR_STRAND_SQUARE_ZERO.md`.

12 Research results and computational evidence

The linear untwisting case connects the family above to the BMW quotient. The thesis exhibits an explicit two-sided identity for X_3 using the zipper, two-strand linear untwisting, and three-strand canonical untwisting relations [2, Theorem 4.1 and Appendix A]. Its denominator is $D = t^4(t+q^2)$; the stated conclusion applies where D is invertible. Corollary 4.2 identifies the resulting quotient with the BMW presentation under the parameter hypotheses of Proposition 3.15. For comparison with the conventional parameters, adjoin a chosen $q^{1/2}$ and assume $q, t, 1-q$ are units; then

$$m = q^{-1/2}(1-q), \qquad l = q^{1/2}t^{-1}, \qquad G_i \longmapsto q^{-1/2}\sigma_i, \quad E_i \longmapsto (1-q)^{-1}X_{2,i}.$$

This scalar extension is part of that parameter comparison.

The current free-algebra certificate example and the thesis's compact braid-group-algebra representation use different ambient equalities. Checking the latter requires braid normal forms or explicit braid/inverse relation terms; it cannot be passed unchanged to the free-algebra checker.

The thesis also discusses dimension, module structure and possible semisimplicity patterns. These questions motivate experiments, but are separate from an individual certificate's conclusion. Neither an interpolation fit nor agreement at selected specializations supplies a generic identity. For the historical reconstruction, it is the final exact identity that supplies the membership proof; stability observed in finitely many sampled reduction shapes is evidence used in finding it.

A finite computation, a small residual or a successful process exit does not certify a completed generic Gröbner basis. Dimension and trace computations require basis-completion and exact-arithmetic evidence for their stated coefficient domain. Solver execution, generic basis certification and the experiment drivers retain separate completion requirements. The general finite-generation theorem gives mathematical context; the cofactor and completed-basis verifiers check their respective explicitly submitted data.

The optional GAP exporter produces a standalone identity in the free associative algebra over $\mathbb{Q}(q,t)$. It shares the validated certificate parser and defining presentation with the Python implementation; GAP separately performs the rational-function arithmetic and noncommutative expansion. A zero residual checks the generic identity, subject to the denominator hypotheses above, without a Gröbner-basis computation.

13 Verification commands and source map

The solver-free regressions and distribution checks cover strict certificate validation, noncommuting contexts, repeated basis occurrences, schedule alignment, poles, target degrees and current certificate interchange formats. See `docs/TESTING.md` for the maintained commands and optional integrations.

The maintained implementation is under `src/zipper_algebra/`:

- Presentation and relation formulas: `presentation.py`, `relations.py`.
- Parameter fields and free algebras: `fields.py`, `algebra.py`.
- Finite word bases and regular actions: `verification/finite.py`.
- Shared ideal derivations: `verification/derivations.py`.
- Internal working arithmetic: `verification/working_field.py`.
- Complete bases and normal forms: `verification/groebner.py`.
- Numeric solver-certificate decoding: `verification/f4ncgb.py`.
- Certificate data: `cofactor/cofactor_io.py`, `cofactor/atoms.py`.
- Symbolic and grid checks: `cofactor/verify.py`.
- Independent GAP identity expansion: `verification/gap.py`.
- Proof graph and reduction: `cofactor/proof.py`, `cofactor/skeleton.py`.
- Symbolic composition: `cofactor/compose.py`.
- Coefficient reconstruction: `cofactor/lift.py`.

The installed discovery command uses `execution/pipeline.py`. Its explicit configuration, proof validation, versioned artifacts and atomic output records are documented in `WORKFLOW.md`.

References

[1] Stephen Bigelow, *Braid groups and Iwahori–Hecke algebras*, in *Problems on Mapping Class Groups and Related Topics*, Proceedings of Symposia in Pure Mathematics **74**, AMS, 2006, pp. 285–299. In particular, §8, Questions 6–8. Author's copy; arXiv:math/0505064.

[2] Choomno Moos, *The Zipper Algebra*, M.A. thesis, University of California, Santa Barbara, September 2026. Cited by section and theorem number for its definitions and research results.

[3] Joan S. Birman and Hans Wenzl, *Braids, link polynomials and a new algebra*, Transactions of the American Mathematical Society **313** (1989), no. 1, pp. 249–273. doi:10.1090/S0002-9947-1989-0992598-X.

[4] Jun Murakami, *The Kauffman polynomial of links and representation theory*, Osaka Journal of Mathematics **24** (1987), no. 4, pp. 745–758.

[5] Maximilian Heisinger and Clemens Hofstadler, *f4ncgb: High Performance Gröbner Basis Computations in Free Algebras*, in *Computer Algebra in Scientific Computing (CASC 2025)*, Lecture Notes in Computer Science **16235**, Springer Nature Switzerland, 2025, pp. 79–97. doi:10.1007/978-3-032-09645-6_5.

[6] George M. Bergman, *The diamond lemma for ring theory*, Advances in Mathematics **29** (1978), no. 2, pp. 178–218. doi:10.1016/0001-8708(78)90010-5; Author's corrections and updates.

[7] A. M. Cohen and J. W. Knopper, *GBNP*, version 1.1.0, GAP package. Official function manual.

[8] Wolfram Decker, Gert-Martin Greuel, Gerhard Pfister and Hans Schönemann, *Singular*, version 4.4.1, computer algebra system. Official project site; Citation information.

[9] Tiziano Peraro, *FiniteFlow: multivariate functional reconstruction using finite fields and dataflow graphs*, Journal of High Energy Physics **2019** (2019), no. 7, article 031. doi:10.1007/JHEP07(2019)031; arXiv:1905.08019. The maintained reconstruction interface uses the FiniteFlow fork; the original framework is upstream FiniteFlow.

[10] Aaron Meurer et al., *SymPy: symbolic computing in Python*, PeerJ Computer Science **3** (2017), e103. doi:10.7717/peerj-cs.103; Official citation metadata; Rational-function domain documentation.